Persistencia local con Room

Proyecto de partida

- Para esta presentación vamos a comenzar desde un proyecto base.
- Para disponer del proyecto base, tenemos que:
  - Crear un proyecto nuevo con la plantilla de Empty Activity.
  - En el fichero MainActivity.kt del proyecto integrar el código del MainActivity.kt dispuesto en una diapositiva posterior.
  - Crear el fichero de clase TareasViewModel.kt y en él integrar el código del TareasViewModel.kt dispuesto en una diapositiva posterior.
  - Crear el fichero de kotlin TareasScreen.kt y en él integrar el código del TareasScreen.kt dispuesto en una diapositiva posterior.
  - En el fichero build.gradle.kts (el de módulo app) añadir la dependencia “implementacion(“androidx.lifecycle:lifecycle-viewmodel-compose:2.10.0”)” y después sincronizar el proyecto con los ficheros Gradle.

MainActivity.kt

```
package eus.fptxurdinaga.pmdm.p06_d00

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.lifecycle.viewmodel.compose.viewModel

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            val tareasViewModel: TareasViewModel =
                viewModel()

            TareasScreen(
                tareas = tareasViewModel.tareas,
                onInsertar = tareasViewModel::insertar
            )
        }
    }
}
```

TareasViewModel.kt

```
package eus.fptxurdinaga.pmdm.p06_d00

import androidx.compose.runtime.mutableStateListOf
import androidx.lifecycle.ViewModel

class TareasViewModel : ViewModel() {

    private val _tareas = mutableStateListOf<String>()

    val tareas: List<String>
        get() = _tareas

    fun insertar(texto: String) {
        _tareas.add(texto)
    }
}
```

TareasScreen.kt (1/3)

```
package eus.fptxurdinaga.pmdm.p06_d00

import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Button
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp

@Composable
fun TareasScreen(
    tareas: List<String>,
    onInsertar: (String) -> Unit
) {
```

TareasScreen.kt (2/3)

```
var texto by remember {
    mutableStateOf("")
}

Column(
    modifier = Modifier.padding(24.dp)
) {

    OutlinedTextField(
        value = texto,
        onValueChange = { nuevoTexto ->
            texto = nuevoTexto
        },
        label = {
            Text("Nueva tarea")
        },
        singleLine = true
    )

    Spacer(
        modifier = Modifier.height(8.dp)
    )
```

TareasScreen.kt (3/3)

```
        Button(
            onClick = {
                if (texto.isNotBlank()) {
                    onInsertar(texto)
                    texto = ""
                }
            }
        ) {
            Text("Añadir")
        }

        Spacer(
            modifier = Modifier.height(24.dp)
        )

        Text("Tareas")

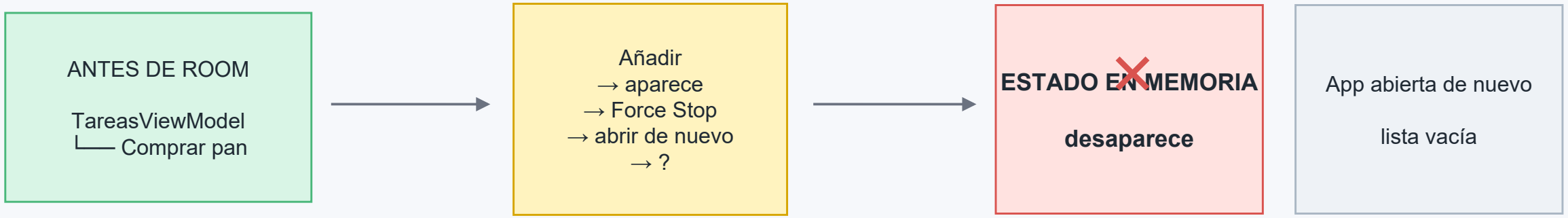
        for (tarea in tareas) {
            Text("• $tarea")
        }
    }
}
```

El dato estaba… y desaparece

Persistencia local con Room

PMDM · 2.º DAM

Persistencia local con Room



Pregunta de la presentación anterior

¿Qué ocurre cuando necesitamos que el dato sobreviva también a la desaparición del estado en memoria?

```
ESTADO EN MEMORIA
        ↓
   desaparece

¿qué datos necesitamos recuperar después?
```

Estado y persistencia resuelven problemas distintos

remember
→ estado local ligado a una composición

ViewModel
→ estado/lógica asociados
a un propietario y un ámbito

Room
→ persistencia local estructurada

No confundir

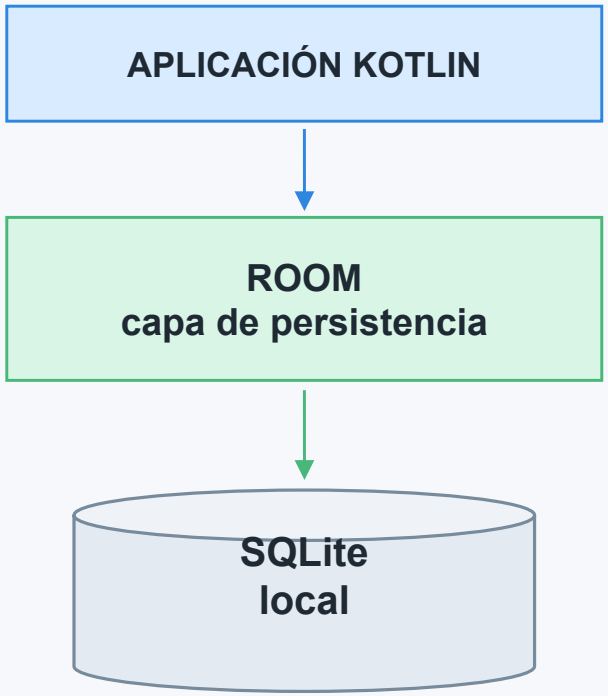
remember ≠ ViewModel ≠ persistencia

Dato	Tratamiento
texto que estoy escribiendo	estado local
mensaje temporal de error	estado local
estado de un flujo/pantalla	ViewModel cuando corresponde
tareas que quiero encontrar mañana	persistencia

Pregunta decisiva

¿Necesito recuperar este dato
después de perder el estado en memoria?

Room trabaja sobre SQLite



Room aporta

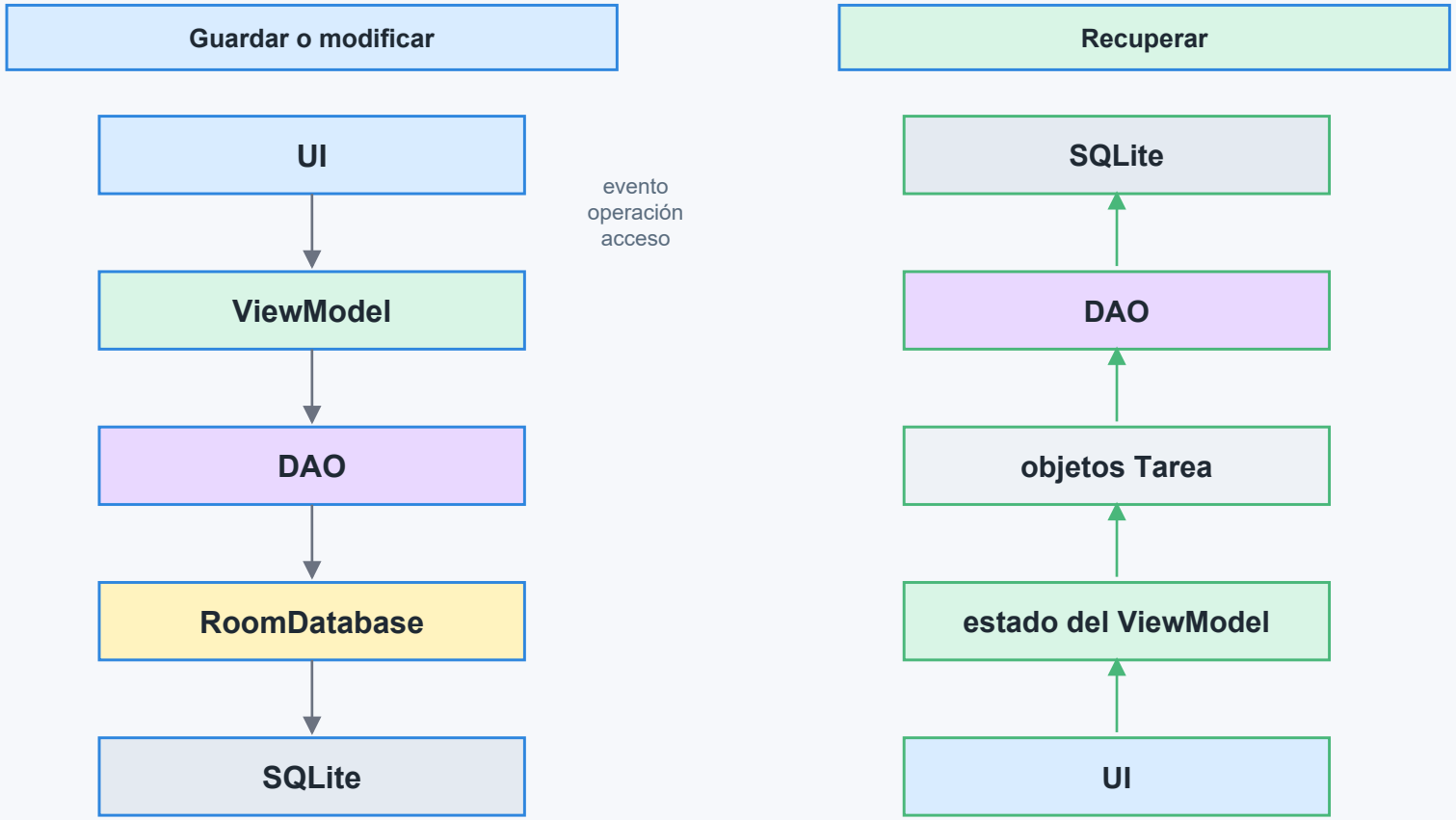
```
objetos Kotlin ↔ datos
DAO → operaciones de acceso
@Query → SQL comprobable al compilar
menos código repetitivo
```

Persistencia local estructurada
No sincronización remota

No confundir

Room ≠ otro motor de BD diferente de SQLite
Room ≠ nube

El mapa completo: de la UI a SQLite y de vuelta



Cada pieza tiene una responsabilidad.

```
UI → representa y comunica eventos
ViewModel → coordina estado y operaciones
DAO → define las operaciones sobre los datos
RoomDatabase → define la base de datos como
conjunto y da acceso a sus DAO
SQLite → almacenamiento relacional subyacente
```

Sin Repository, Hilt ni capas extra en presentación.

Configurar Room: KSP, dependencias, Sync y Build

Room 3.0.2 · androidx.room3 · KSP

1. Comprobar la versión Kotlin/KGP del proyecto en gradle/libs.versions.toml buscando:

```
kotlin = "A.B.C"
```

2. Elegir una versión de KSP compatible con el proyecto. KSP 2.3.10 admite KGP 2.2.10-2.3.X, pero además exige AGP >= 8.12.0, Gradle >= 8.13 y JDK >= 17, por lo que hay que comprobarlos.

2.1. En gradle/libs.versions.toml, buscar para contrastar con las exigencias:

```
agp = "A.B.C"
```

2.2. En gradle/wrapper/gradle-wrapper.properties buscar para contrastar con las exigencias:

```
gradle-A.B.C-bin.zip
```

2.3. En la terminal de Android Studio ejecutar:

```
.\gradlew.bat --version
```

y buscar para contrastar con las exigencias:

```
JVM = A.B.C
```

3. Declarar la versión de KSP en gradle/libs.versions.toml. Concretamente, añadir en la sección de [versions]:

```
ksp = "2.3.10"
```

4. Declarar el plugin KSP en el catálogo de versiones en gradle/libs.versions.toml. Concretamente, añadir en la sección de [plugins]:

```
ksp = { id = "com.google.devtools.ksp", version.ref = "ksp"}
```

En el proyecto de clase

versión de Kotlin
↕ compatible
versión de KSP

¿Para qué está KSP aquí?

```
@Entity · @Dao · @Database
          ↓
     KSP procesa
          ↓
Room genera código
          ↓
        Build
```

Después

Gradle Sync
→
Build
→
continuar

Acción guiada
1 aplicar plugin
2 añadir dependencias
3 Sync
4 Build

Configurar Room: KSP, dependencias, Sync y Build

Room 3.0.2 · androidx.room3 · KSP

5. Aplicar el plugin KSP al módulo app vía su build.gradle.kts, añadiendo en el bloque de plugins:

```
alias(libs.plugins.ksp)
```

6. Añadir Room al módulo app vía su build.gradle.kts, añadiendo en el bloque de dependencies:

```
val = roomVersion = "3.0.2"

implementation(
    "androidx.room3:room3-runtime:$roomVersion"
)

ksp(
    "androidx.room3:room3-compiler:$roomVersion"
)
```

7. Ejecutar Gradle Sync vía "Sync Project with Gradle Files".
8. Ejecutar Build Project y comprobar que no hay errores.

En el proyecto de clase

versión de Kotlin
↕ compatible
versión de KSP

¿Para qué está KSP aquí?

```
@Entity · @Dao · @Database
          ↓
     KSP procesa
          ↓
Room genera código
          ↓
        Build
```

Después

Gradle Sync
→
Build
→
continuar

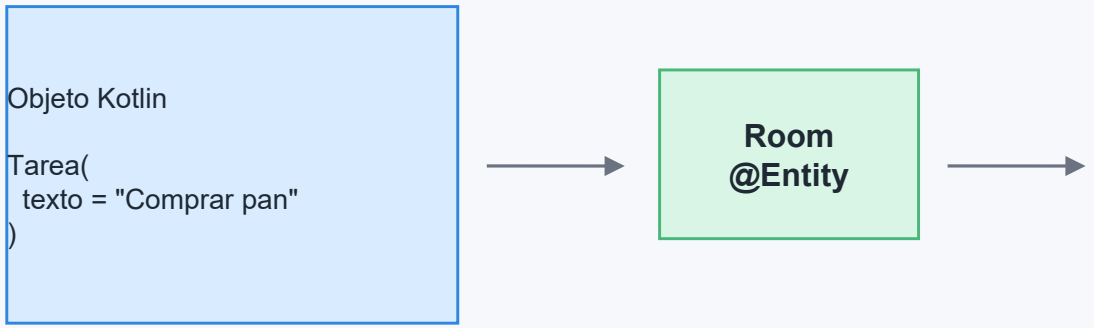
Acción guiada
1 aplicar plugin
2 añadir dependencias
3 Sync
4 Build

Una Tarea: objeto Kotlin y fila persistente

Tarea.kt

```
import androidx.room3.Entity
import androidx.room3.PrimaryKey

@Entity(tableName = "tareas")
data class Tarea(
    @PrimaryKey(autoGenerate = true)
    val id: Int = 0,
    val texto: String,
    val hecha: Boolean = false
)
```



id	texto	hecha
1	Comprar pan	false

tabla tareas

Precisión

crear un objeto Tarea ≠ insertarlo en SQLite

id = 0 al insertar
→ Room/SQLite asignan la clave.

El DAO define las operaciones de acceso

TareaDao.kt

```
import androidx.room3.Dao
import androidx.room3.Insert
import androidx.room3.Query

@Dao
interface TareaDao {

    @Insert
    suspend fun insertar(tarea: Tarea)

    @Query(
        "SELECT * FROM tareas ORDER BY id"
    )
    suspend fun obtenerTodas(): List<Tarea>
}
```

ViewModel
↓ solicita
TareaDao
↓ define acceso
Room / SQLite

La consulta

```
SELECT * FROM tareas ORDER BY id
```

```
tabla: tareas
resultado: List<Tarea>
```

suspend visible
Lo explicamos cuando aparezca la necesidad.

CRUD: cuatro capacidades sobre las mismas tareas

TareaDao.kt

```
import androidx.room3.Dao
import androidx.room3.Delete
import androidx.room3.Insert
import androidx.room3.Query
import androidx.room3.Update

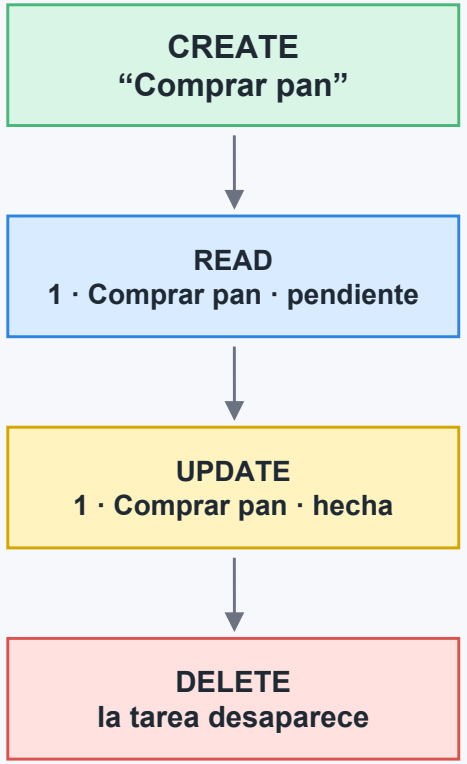
@Dao
interface TareaDao {

    @Insert
    suspend fun insertar(tarea: Tarea)

    @Query(
        "SELECT * FROM tareas ORDER BY id"
    )
    suspend fun obtenerTodas(): List<Tarea>

    @Update
    suspend fun actualizar(tarea: Tarea)

    @Delete
    suspend fun eliminar(tarea: Tarea)
}
```




```
Create → @Insert
Read   → @Query
Update → @Update
Delete → @Delete
```

La misma tarea
La identidad real viene de Room.

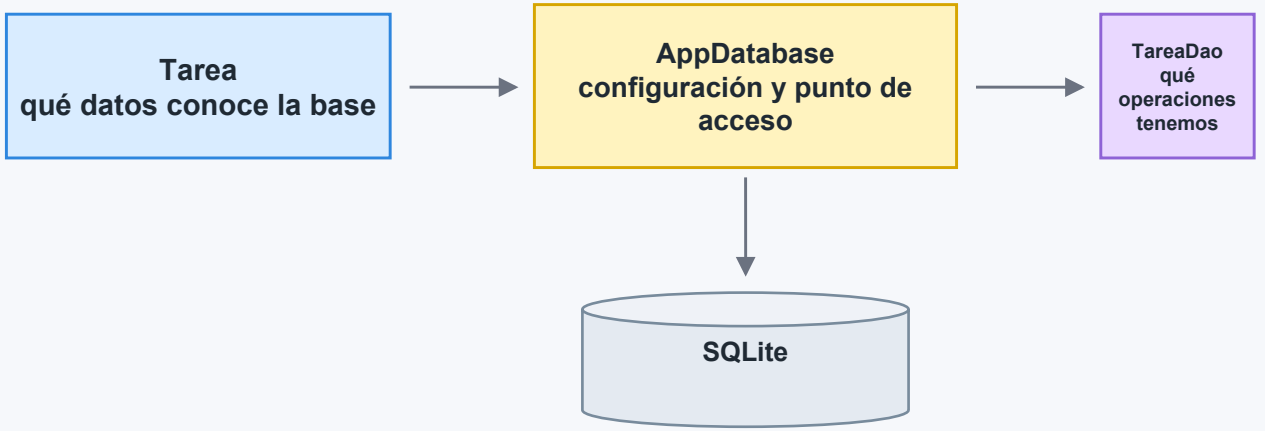
RoomDatabase conoce las entidades y proporciona el DAO

AppDatabase.kt

```
import androidx.room3.Database
import androidx.room3.RoomDatabase

@Database(
    entities = [Tarea::class],
    version = 1
)
abstract class AppDatabase : RoomDatabase() {

    abstract fun tareaDao(): TareaDao
}
```



Ahora

version = 1

No modificamos el esquema durante la práctica.
Migraciones → fuera de esta presentación.

La base no se construye dentro de la UI

PmdmApplication.kt

```
import android.app.Application
import androidx.room3.Room
import androidx.sqlite.driver.AndroidSQLiteDriver

class PmdmApplication : Application() {

    val database: AppDatabase by lazy {
        Room.databaseBuilder<AppDatabase>(
            applicationContext,
            "pmdm.db"
        )
            .setDriver(
                AndroidSQLiteDriver()
            )
            .build()
    }
}
```

AndroidManifest.xml

```
<application

android:name=".PmdmApplication"
    ... >
```

```
proceso actual
└── una instancia estable de
AppDatabase
          ↓
      pmdm.db
```

No hacer

@Composable
→ Room.databaseBuilder(...)
→ crear otra base por recomposición

```
PmdmApplication
└── database
    └── AppDatabase
        └── pmdm.db
```

El ViewModel necesita el DAO (1/2)

TareasViewModel.kt

```
import androidx.lifecycle.ViewModel

class TareasViewModel(
    private val dao: TareaDao
) : ViewModel() {

    // estado y operaciones
}
```

PmdmApplication
→ AppDatabase
→ TareaDao
→ TareasViewModel

Problema nuevo

En la presentación “Estado compartido, ViewModel y navegación entre pantallas” el ViewModel no necesitaba dependencias externas.

Ahora necesita un DAO.

Idea que hay que comprender

“este ViewModel necesita este DAO”

No se resuelve creando manualmente el ViewModel dentro de un composable.

El ViewModel necesita el DAO (2/2)

TareasViewModel.kt

```
import androidx.lifecycle.ViewModelProvider
import androidx.lifecycle.ViewModelProvider
    .AndroidViewModelFactory
    .Companion
    .APPLICATION_KEY
import androidx.lifecycle.viewmodel.initializer
import androidx.lifecycle.viewmodel.viewModelFactory

companion object {

    val Factory: ViewModelProvider.Factory =
        viewModelFactory {
            initializer {
                val app =
                    this[APPLICATION_KEY]
                        as PmdmApplication

                TareasViewModel(
                    app.database.tareaDao()
                )
            }
        }
}
```

En Compose:

```
import
androidx.lifecycle.viewmodel.compose.viewModel

val tareasViewModel: TareasViewModel =
    viewModel(
        factory = TareasViewModel.Factory
    )
```

Factory
→ obtiene Application
→ accede a la base
→ obtiene DAO
→ construye ViewModel

No memorizar toda la API de CreationExtras.
Comprender: “este ViewModel necesita este DAO”.

No sustituir por TareasViewModel(dao)
dentro de la UI.

suspend aparece porque el DAO no debe bloquear la UI

Tenemos:

```
suspend fun obtenerTodas(): List<Tarea>
```

El ViewModel necesita llamarla:

```
import androidx.lifecycle.viewModelScope
import kotlinx.coroutines.launch

viewModelScope.launch {
    val datos =
        dao.obtenerTodas()
}
```

Modelo operativo

ViewModel necesita operación DAO
↓
la función es suspend
↓
viewModelScope.launch { ... }
↓
se realiza la operación soportada por Room

No confundir

suspend
≠ “crear un hilo secundario”

viewModelScope
≠ “hilo de BD”

corrutina
≠ hilo

Sin Dispatchers como tema nuevo en esta presentación.

Al crear el ViewModel, cargamos lo persistido

TareasViewModel.kt — Imports ya introducidos omitidos por legibilidad.

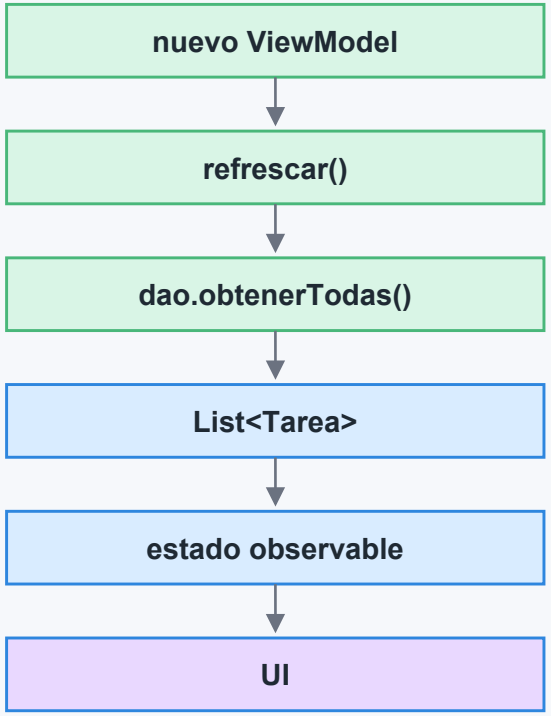
```
private val _tareas =
    mutableStateListOf<Tarea>()

val tareas: List<Tarea>
    get() = _tareas

init {
    viewModelScope.launch {
        refrescar()
    }
}

private suspend fun refrescar() {
    val datos =
        dao.obtenerTodas()

    _tareas.clear()
    _tareas.addAll(datos)
}
```



Actualización explícita
No utilizamos Flow en esta V1.

Insertar significa persistir y después refrescar

TareasViewModel.kt

```
fun insertar(texto: String) {
    viewModelScope.launch {

        dao.insertar(
            Tarea(
                texto = texto
            )
        )

        refrescar()
    }
}
```

```
UI
↓ “Comprar pan”
ViewModel.insertar()
dao.insertar(...)
fila persistida
refrescar()
dao.obtenerTodas()
estado actualizado
UI actualizada
```

guardar en la BD
≠
actualizar automáticamente nuestro estado explícito

La UI verá la entidad real recuperada desde Room, con su identificador.

Actualizar y eliminar usan la identidad recuperada

TareasViewModel.kt

```
fun cambiarEstado(tarea: Tarea) {
    viewModelScope.launch {

        dao.actualizar(
            Tarea(
                id = tarea.id,
                texto = tarea.texto,
                hecha = !tarea.hecha
            )
        )

        refrescar()
    }
}
```

```
fun eliminar(tarea: Tarea) {
    viewModelScope.launch {
        dao.eliminar(tarea)
        refrescar()
    }
}
```

Tarea recuperada
id real
↓
UPDATE / DELETE
↓
misma fila
↓
refrescar()

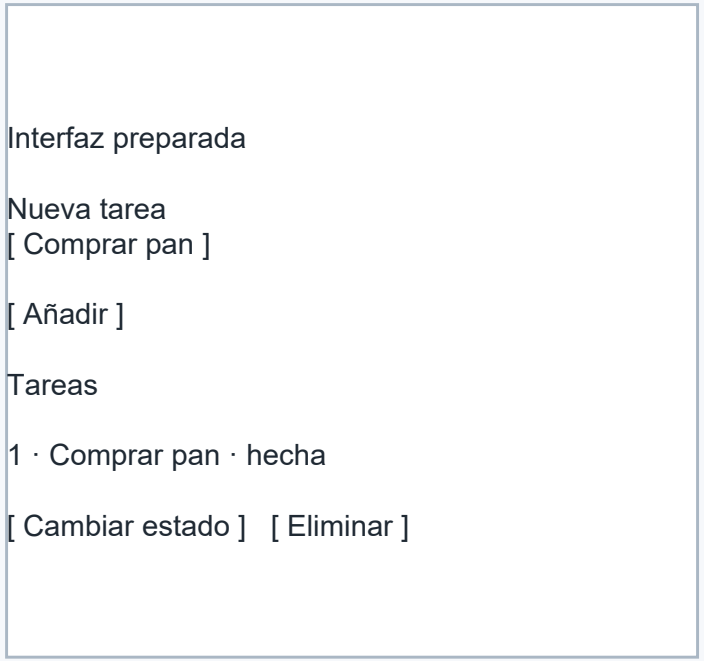
Regla

No inventamos un id.
Utilizamos la entidad
que ya hemos recuperado.

Compose representa; el ViewModel coordina; el DAO accede

Conexión:

```
TareasScreen(
    tareas = tareasViewModel.tareas,
    onInsertar =
        tareasViewModel::insertar,
    onCambiarEstado =
        tareasViewModel::cambiarEstado,
    onEliminar =
        tareasViewModel::eliminar
)
```




```
Responsabilidades

UI
✓ representa
✓ comunica eventos

ViewModel
✓ coordina estado y operaciones

DAO
✓ accede a datos

UI
✕ no crea RoomDatabase
✕ no ejecuta SQL
```

Ahora sí: demostramos persistencia

Prueba 1 — el dato sobrevive

```
GUARDAR
Comprar pan · hecha
        ↓
FORCE STOP
        ↓
ABRIR DESDE EL LAUNCHER
        ↓
nuevo proceso
nuevo ViewModel
        ↓
refrescar()
        ↓
Room / SQLite
        ↓
Comprar pan · hecha
```

Prueba 2 — el borrado también persiste

```
Eliminar
   ↓
Force Stop
   ↓
abrir de nuevo
   ↓
Comprar pan NO aparece
```

No demuestra lo mismo

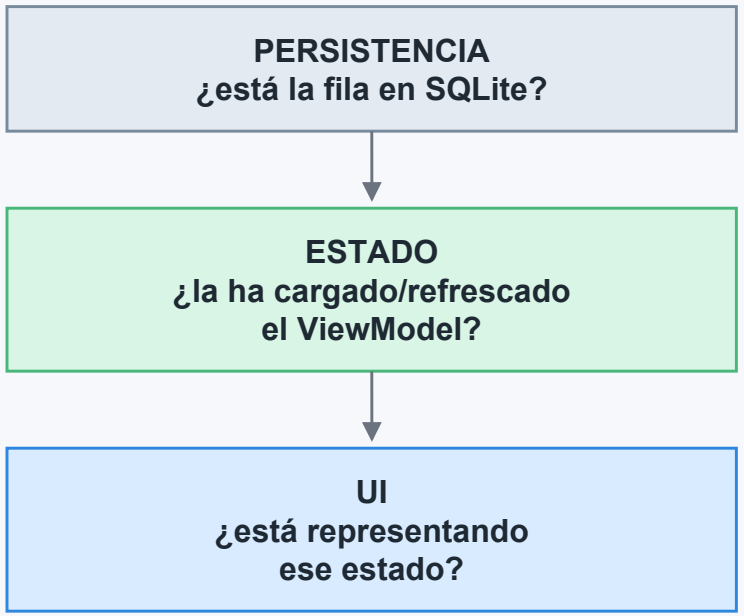
Back
navegar
recomponer
pulsar Home

No utilizar como prueba

Clear data
Clear storage
desinstalar

Eso elimina el almacenamiento de la aplicación.

«No aparece» no significa «no está guardado»



Puede ocurrir:

```
SQLite
✓ tarea guardada

ViewModel
✕ lista sin refrescar

UI
✕ tarea no visible
```

Diagnóstico

no aparece
≠
no se guardó

Seguir el flujo.
No adivinar.

Diagnosticar Room sin probar cosas al azar

Síntoma	Revisar primero
Entity, Dao o ksp(...) no se reconocen	plugin · dependencias · Sync
no se genera/compila lo esperado	KSP · Build
la base no conoce Tarea	entities = [Tarea::class]
falla @Query al compilar	tabla · columnas · retorno
no puedo llamar a función suspend	contexto de llamada
BD cambia pero UI no	refrescar() · estado · UI

Tres momentos diferentes

```
CONFIGURACIÓN
→ Sync

CÓDIGO
→ Build

COMPORTAMIENTO
→ Run / observar
```

Ciclo

localizar
→ corregir
→ Sync/Build/Run según corresponda
→ observar

Persistencia local resuelta

Hoy hemos resuelto

estado en memoria
↓
persistencia local

```
UI
↕ estado / eventos
ViewModel
↕ operaciones
DAO
↕ acceso
RoomDatabase
↕
SQLite
```

Ya podemos

crear
consultar
modificar
eliminar
terminar el proceso
volver a abrir
recuperar

Modelo final

remember
→ estado local de UI

ViewModel
→ estado/lógica con propietario y ámbito

Room
→ persistencia local estructurada

De Android Studio a una primera aplicación con Jetpack Compose → UI
Kotlin aplicado: interacción y estado en Compose → interacción + estado
Estado compartido, ViewModel y navegación entre pantallas → ViewModel + navegación
Esta presentación → persistencia
Próxima presentación → integración

Pregunta para la próxima presentación

Ya sabemos construir UI, gestionar estado, navegar y persistir.
¿Podemos integrarlo, probarlo sistemáticamente y demostrar que nuestra aplicación está preparada para el reto?